

SYSTEMANALYSE UND ENTWURF

CineTicket

Ein Kino-Ticketsystem. Von der Anforderungsanalyse über die UML-Modelle bis zum klickbaren Prototyp.

So ist das Projekt gebaut

Alle Inhalte sind strukturierte Daten

- Personas, Stories, UML und Zustände liegen als JSON-Dateien vor
- Die Diagramme werden daraus live als SVG gerendert
- Inhalt und Darstellung können nicht auseinanderlaufen

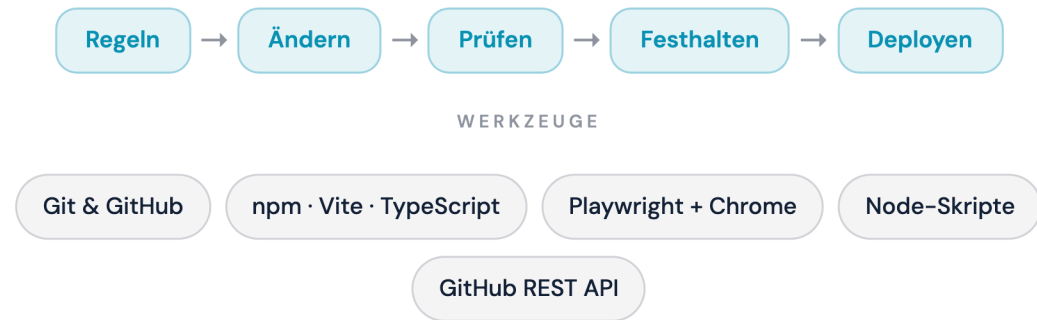


Zwei Detailgrade auf der Website: Einfach ist immer eine echte Teilmenge von Erweitert.

So ist es entstanden

Fester Ablauf, Standard-Werkzeuge

- Jede Änderung: Regeln lesen, ändern, prüfen, festhalten, deployen
- Playwright testet die echte Seite im Browser
- Node-Skripte prüfen die Datenkonsistenz



Nichts geht ungeprüft live: Lint, Build und Browser-Test sichern jeden Stand.

Der Weg: drei Stationen

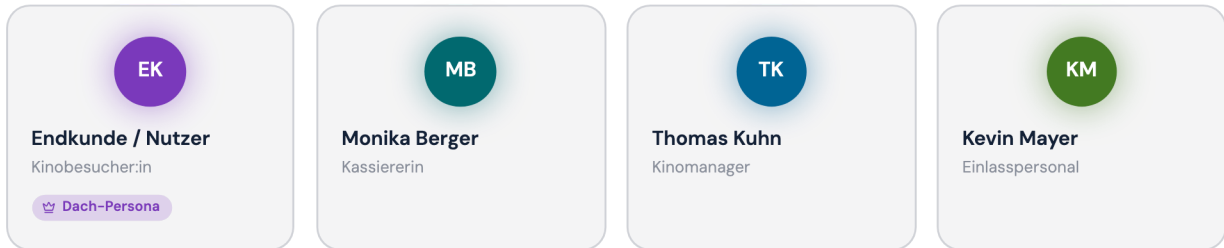
Jede Station baut auf der vorigen auf. Ideen über den MVP hinaus schließen den Bogen.



Kundschaft und Betrieb

Eine Rolle draußen, drei drinnen

- Der Endkunde bucht Tickets, mobil oder im Web
- Im Betrieb: Monika an der Kasse, Thomas als Manager, Kevin am Einlass
- Verschiedene Ziele ergeben verschiedene Anforderungen



Jede User Story gehört zu genau einer Persona.

So tickt die Kundschaft

Schnell buchen, den Lieblingsplatz sichern

- Frustrationen: lange Ladezeiten, Gruppen-Tickets nur einzeln buchbar
- Daraus entstehen der Buchungs-Wizard und der Sitz-Hold

**Endkunde / Nutzer**
Kinobesucher:in · 35 Jahre

„In 60 Sekunden gebucht, der Lieblingsplatz reserviert.“

 ZIELE	 FRUSTRATIONEN
• Schnell und unkompliziert Tickets kaufen (mobil unter 60 Sekunden) • Den besten freien Platz per KI-Vorschlag mit einem Tipp übernehmen • Rabatte (Senior, Student, Kind, Gruppe) ohne Hürden anwenden • Barrierefreie Plätze und Begleitperson-Optionen klar erkennen • Bonuspunkte sammeln, einsehen und einlösen	• Lange Ladezeiten und komplizierte Mobile-Buchung • Sitzplan ohne Komfort- oder Barrierefreiheits-Details • Gruppen-Tickets nur einzeln buchbar • Keine modernen Zahlungsmittel (Apple Pay / Google Pay)

So tickt der Betrieb

An der Kasse zählt Tempo

- Monika will schnell und fehlerfrei verkaufen
- Frustration: langsame Systeme im Stoßbetrieb
- Daraus entsteht der Schnellverkauf mit Tastatur-Shortcuts



Monika Berger

Kassiererin · 38 Jahre

„Schnell, korrekt, freundlich – in dieser Reihenfolge.“

🎯 ZIELE

- Tickets schnell und fehlerfrei verkaufen
- Warteschlangen minimieren
- Einfache Stornierungen durchführen

😞 FRUSTRATIONEN

- Langsame Ladezeiten im System
- Komplizierte Sitzplatzbuchung unter Zeitdruck
- Keine Übersicht bei ausverkauften Vorstellungen

Im Vollausbau werden aus den Rollen konkrete Profile, von der Stammkundin bis zur Familienmutter.

Ein Satz, drei Antworten

Wer braucht was, und wozu?

- Als Persona möchte ich Ziel, damit Nutzen
- Akzeptanzkriterien machen testbar, wann eine Story fertig ist

Als **Persona** möchte ich **Ziel** um **Nutzen** .

Persona – wer braucht es?

Ziel – was soll möglich sein?

Nutzen – warum lohnt es sich?

Die wichtigste Regel: der Sitz-Hold

U47 verhindert Doppelbuchungen

- Beim Checkout wird der Platz verbindlich reserviert
- Priorität hoch, eingeplant für Release 1
- Taucht wieder auf: im Sequenzdiagramm und im Zustandsautomaten

U47

Hohe Priorität

Release 1 – MVP

LH Lara

Als Stammkundin möchte ich, dass mein gewählter Platz beim Checkout verbindlich für eine begrenzte Zeit reserviert wird, damit ihn niemand parallel buchen kann (keine Doppelbuchung).

AKZEPTANZKRITERIEN

- ✓ Platz wird zu Beginn des Checkouts auf Status RESERVIERT gesetzt
- ✓ Hold läuft nach 10 Minuten automatisch ab und gibt den Platz wieder frei
- ✓ Eine parallele Buchung desselben Platzes wird abgewiesen
- ✓ Erst die bezahlte Buchung setzt den Platz auf BELEGT

Priorität und Release: der Bauplan

Priorität und Release ergeben den Bauplan

- Jede Story trägt eine MoSCoW-Priorität und ein Release
- Release 1 ist der MVP
- Spätere Releases erweitern dort, wo es Wert stiftet

30 Stories im Einfach-Modus

↔

51 im Erweitert-Modus

NACH RELEASE

16 Release 1

27 Release 2

8 Release 3

NACH PRIORITÄT (MOSCOW)

22 Hohe

22 Mittlere

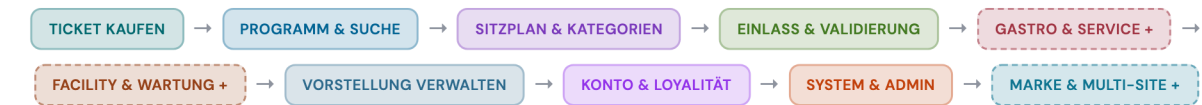
7 Niedrige

Jede Story kennt Persona, Aktivität und Release. Nichts hängt in der Luft.

Das Backbone: die Nutzerreise

Waagrecht die Nutzerreise, senkrecht die Releases

- Das Rückgrat: Aktivitäten vom Ticketkauf bis zur Verwaltung
- Der Vollausbau ergänzt Gastro, Facility und Multi-Site



„+“ = Aktivität kommt erst im Erweitert-Modus dazu (7 → 10)

Waagerecht schneiden: Releases

Release 1: die schmale, lauffähige Scheibe

- Kern-Aktivitäten: kaufen, Sitz wählen, einlösen, verwalten
- Suche, Gastro, Facility und Marke folgen bewusst später
- Erst das Nötige ganz, statt überall ein bisschen

ALLE AKTIVITÄTEN →

Release 1 – MVP die schmale, lauffähige Scheibe — ihren Kern zeigt der Prototyp **16 Stories**

Release 2 – Erweiterung Komfort, Gastro & Service, Facility **27 Stories**

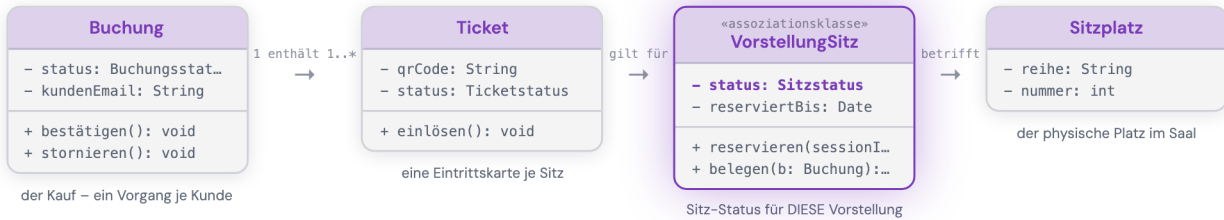
Release 3 – Vollausbau Empfehlungen, Reports, Multi-Site **8 Stories**

Auf der Website ist die Karte interaktiv: Bänder hervorheben, jede Karte öffnet ihre Story.

Das Herzstück: eine Buchung

Vier Klassen, vier klare Aufgaben

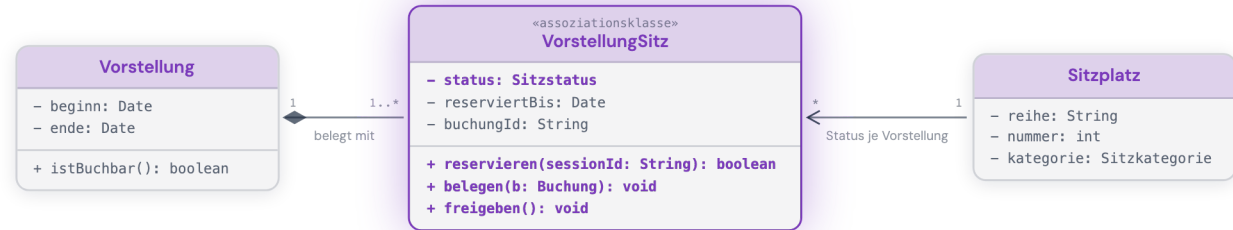
- Die Buchung ist der Kauf und enthält je Platz ein Ticket
- Jedes Ticket gilt für einen VorstellungSitz
- Der physische Sitzplatz im Saal bleibt unberührt



Der Schlüssel: VorstellungSitz

Eine Assoziationsklasse trennt Sitz und Status

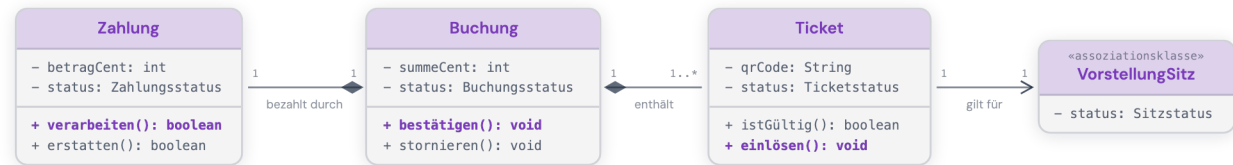
- Ein Sitzplatz existiert dauerhaft, sein Status wechselt je Vorstellung
- reservieren(), belegen() und freigeben() setzen den Hold durch
- reserviertBis steuert später den Timeout



Wer macht was: Objekt vs. Service

Objekte wissen, der Service steuert

- Zahlung.verarbeiten(),
Buchung.bestätigen(), Ticket.einlösen()
- Der BuchungService ruft nur auf und hält
keine Daten
- Buchung ist nicht der BuchungService

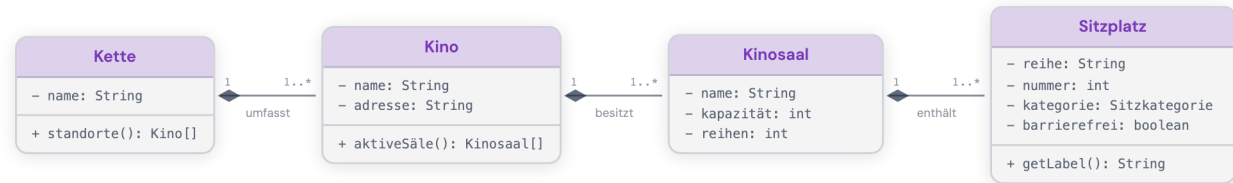


«control» steuert, «entity» weiß.

Vom Saal zum einzelnen Sitz

Der Teil existiert nicht ohne das Ganze

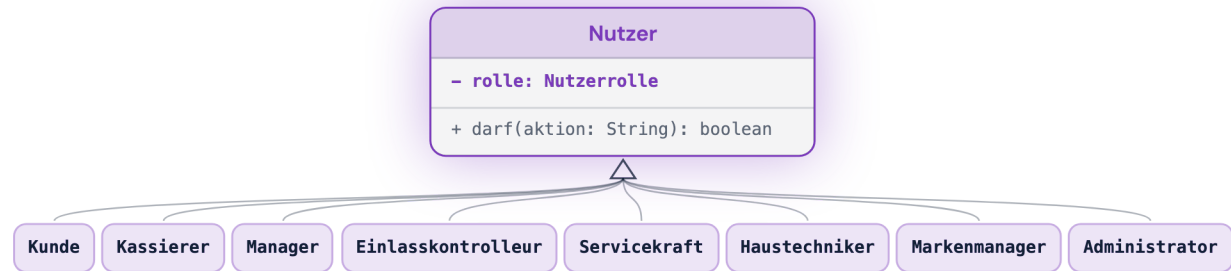
- Eine Kette umfasst Kinos, ein Kino besitzt Säle, ein Saal seine Sitze
- Die gefüllte Raute steht für Komposition



Rollen erben von Nutzer

Alle Rollen erben von Nutzer

- Vom Kunden bis zum Administrator
- Rechte entscheidet das Enum Nutzerrolle
- Geprüft an einer Stelle: darf(aktion)



Der Buchungsablauf über die Zeit

Fokussierte Abläufe statt eines überladenen Diagramms

- Online-Buchung, Kasse, Storno und Einlass
- Die nächsten Folien zoomen in die Online-Buchung

Online-Buchung (Kunde)

Vorstellung → Sitzplan → Hold → Zahlung → QR-Ticket

Kassenverkauf / POS (Kassiererin)

Tickets in den Warenkorb, an der Kasse bezahlen

Storno & Rückerstattung

Buchung stornieren: Ticket ungültig, Zahlung erstattet, Sitz frei

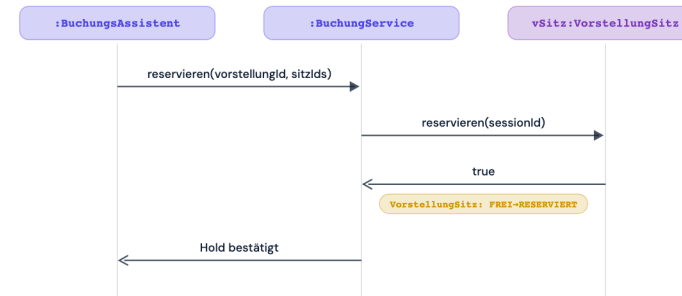
Einlass & Validierung

QR-Code scannen, gültiges Ticket einlösen

Der Hold: 10 Minuten verbindlich

Beim Checkout wird der Sitz serverseitig reserviert

- Der BuchungService ruft reservieren() am VorstellungSitz auf
- Der Status wechselt auf RESERVIERT, parallele Käufer sind blockiert
- Das Badge zeigt den Statuswechsel aus den Daten

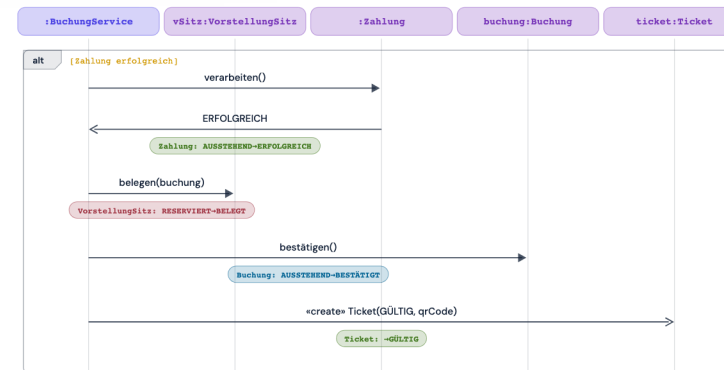


Kommt reservieren() zu spät, antwortet der Sitz mit false. Der Kunde landet in der Sitzwahl, nie in einer Doppelbuchung.

Die Zahlung entscheidet

Erst die Zahlung macht aus dem Hold einen Kauf

- Zahlung erfolgreich, Sitz belegt, Buchung bestätigt, Ticket erzeugt
- Vier Statuswechsel in fester Reihenfolge
- Das alt-Fragment zeigt den bedingten Zweig



Der Storno läuft genauso als Kette: Buchung storniert, Ticket ungültig, Zahlung erstattet, Sitz wieder frei.

Der wichtigste Automat: der Sitz

Ein Automat beschreibt genau ein Objekt über die Zeit

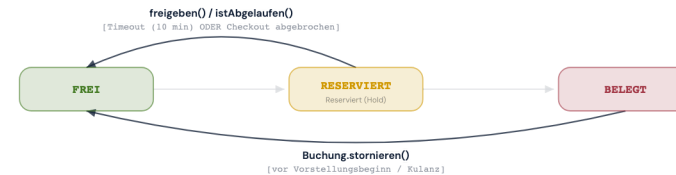
- auswählen() ist nur lokal im Browser
- reservieren() setzt den serverseitigen Hold
- belegen() erst nach erfolgreicher Zahlung



Jeder Hold endet von selbst

Kein Sitz bleibt für immer blockiert

- Hold abgelaufen oder Kauf abgebrochen:
automatisch zurück auf FREI
- Ein Storno gibt auch belegte Plätze frei
- DEFEKT sperrt systemweit und erzeugt ein
Wartungsticket



Jeder Automat ist wertgleich mit seinem Status-Enum im Klassendiagramm.

Die weiteren Automaten in Kürze

Dasselbe Muster für Ticket, Buchung und Zahlung

Ticket-Lebenszyklus

4 Zustände

Ein Anfang, drei Enden: eingelöst, storniert oder abgelaufen.

Buchungs-Lebenszyklus

4 Zustände

Hält alles zusammen. Bestätigt erst mit erfolgreicher Zahlung.

Zahlungs-Lebenszyklus

4 Zustände

Entscheidet den Rest. Erst ein Storno macht daraus ERSTATTET.

Vom Modell zur App

Der Kern des Modells läuft

- Endkunde, Kasse, Manager und Einlass sind klickbar gebaut
- Die übrigen Rollen sind als Roadmap modelliert
- Das Modell beschreibt das ganze Produkt



4 von 8 Rollen und 41 von 82 UML-Klassen sind gebaut.

Modell $\supset$ Prototyp: Was fehlt, ist im Klassendiagramm bereits verortet.

Live-Demo

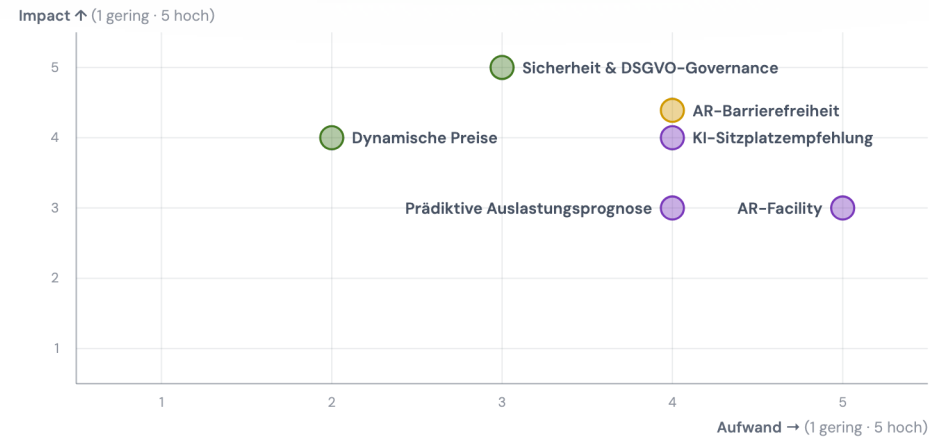
Die echte App, direkt im Browser

- Buchen im Sitzplan, verkaufen an der Kasse
- Steuern im Manager-Cockpit, scannen am Einlass
- Mock-API, keine Installation nötig



Sechs Ideen, nach Wirkung und Aufwand

Impact gegen Aufwand, Machbarkeit als Farbe



● im Rahmen machbar

● teilweise machbar

● Konzept / Vision

Nah am MVP

Drei Ideen docken direkt am System an

Dynamische Preise

im Rahmen machbar

Ticketpreise passen sich regelbasiert an die Auslastung an, um Nachfragespitzen besser zu nutzen.

Sicherheit & DSGVO-Governance

im Rahmen machbar

Least-Privilege-Rollen, lückenloses Audit-Log, erzwungene 2FA sowie DSGVO-Auskunft und -Löschung.

AR-Barrierefreiheit

teilweise machbar

Zu einer Vorstellung lässt sich eine Untertitel- oder Audiodeskriptions-Brille dazubuchen.

Drei Blicke nach vorn

Bewusst als Konzept markiert, recherchiert und eingeordnet

KI-Sitzplatzempfehlung

Konzept / Vision

Die KI schlägt den besten freien Platz passend zu Präferenzen vor – kein langes Suchen im Sitzplan.

AR-Facility: Defekt melden & Sitz sperren

Konzept / Vision

Haustechnik meldet einen defekten Sitz per AR-Brille mit Foto und Sprachnotiz; sofort entsteht ein Wartungsticket und der Sitz wird gesperrt.

Prädiktive Auslastungsprognose

Konzept / Vision

Die erwartete Auslastung kommender Vorstellungen wird prognostiziert, um Programm und Personal vorausschauend zu planen.

Was uns wichtig war

Weiterbauen können: aus der Analyse wird der Bauplan fürs echte System

Daten statt Dokumente

12 Personas, 51 Stories, 82 Klassen als JSON. Daraus entstehen später Schema, Seed-Daten und Testfälle.

Ein roter Faden

U47 führt von der Story über die Sequenz bis zum Automaten. Jede Anforderung kennt Persona und Release.

Keine zwei Wahrheiten

Enums und Automaten sind wertgleich. Die Implementierung kann dem Modell direkt folgen.

Bewiesen und kartiert

4 von 8 Rollen laufen im Prototyp. Der Rest ist im Modell bereits verortet.

Anforderungen → Modell → Prototyp (MVP-Kern) → Release 2–3: das volle System

Vielen Dank. Fragen und Diskussion.